

M10.08 — Graph preview: draw the repository as it *would* be, without touching it

Status: design, awaiting owner review · **Date:** 2026-08-29 ·

Author: max **Depended on by:** [#460](#) (M10.05 plan-review

pane) **Related:** [#457](#) (terminal graph from the lane core), [#92](#) / ADR 0091 (Explain Mode)

1. What this is, in plain language

You are about to run `revert`, or `cherry-pick`, or a merge, and you cannot picture what the graph will look like afterwards. Today the app tells you in words what an operation will do — Explain Mode does that already. It cannot show you.

This draws the *after* graph before you commit to it. Not a simulation of git: real git computes the answer, against your real objects, and writes nothing to your repository.

The idea came from [git-school's Visualizing Git](#), which animates a toy graph as you type commands. The difference that matters: that tool models git, and this one runs it.

2. Goal and acceptance criteria

Goal. Given a `Plan` the planner already produces, return the rows and edges the graph *would* have, plus what changed — as data, renderable by any surface.

- **A1.** `preview(repo, plan)` returns a hypothetical `Vec<GraphRow>` + `Vec<Edge>` laid out by the existing `StreamLayout`, for revert, cherry-pick and merge.
- **A2.** The real repository is **byte-for-byte unchanged**: no new objects, no ref moved, no worktree, no index write. Asserted by a test that counts objects and compares every ref before and after.
- **A3.** A merge that would conflict returns `Conflict { paths }` — a real answer, not an error and not a guessed graph.
- **A4.** An operation the plumbing cannot express returns `Unsupported`. It never returns a graph it is not sure of.
- **A5.** The predicted graph equals the graph produced by actually running the operation in a throwaway clone, over every applicable fixture repository.
- **A6.** The web canvas renders before/after with the changes marked.

3. The finding that shaped the design

Two facts were measured on 2026-08-29 before any of this was designed, and each removed an approach that looked reasonable.

`StreamLayout::push` **takes a commit and a membership predicate — not a repository.** The layout engine never asks whether a commit exists. It will lay out a hypothetical history exactly as it lays out a real one. That is why this feature costs a function rather than a renderer.

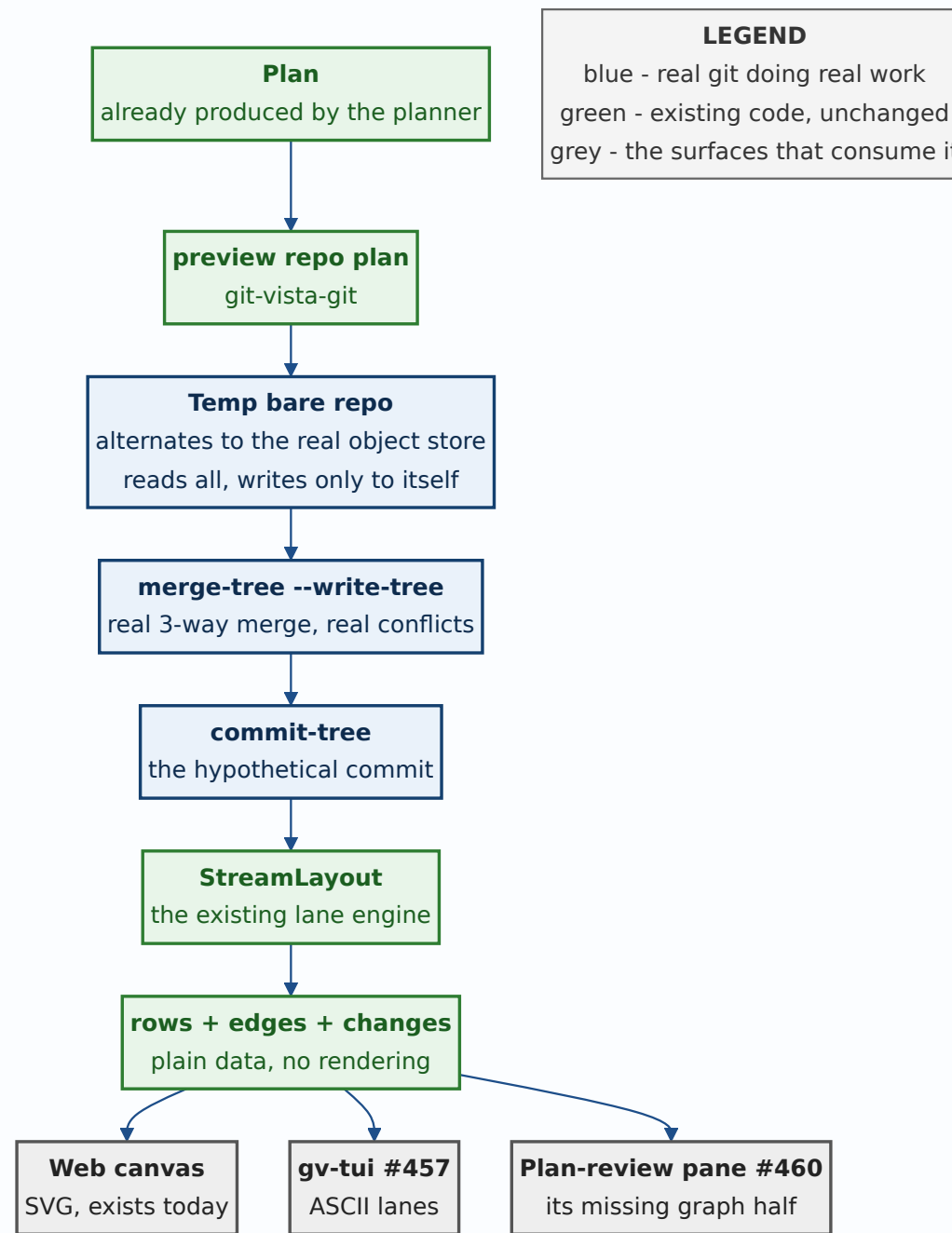
A temp bare repo whose `objects/info/alternates` points at the real object store reads everything and writes only into itself. Measured on this repository: `merge-tree --write-tree` produced tree `35fbd212...`, `commit-tree` produced commit `2b75afa6...`, the temp store could read it back, **the real repository could not**, and its object count was 19,593 before and 19,593 after.

That is the whole safety argument, and it is a measurement rather than a claim.

4. Architecture

4.1 Where the code lives — below the renderer, deliberately

#457 commits the terminal UI to drawing "from the lanes core already computes". A preview computed in the wasm frontend could never serve that surface: it would be written twice and drift twice. So the preview is a function in `git-vista-git`, returning plain data, and every surface is a renderer of it.



4.2 The vocabulary

```
pub fn preview(repo: &Path, plan: &Plan) -> PreviewOutcome;

pub enum PreviewOutcome {
    Graph { rows: Vec<GraphRow>, edges: Vec<Edge>, changes: Vec<Change> },
    Conflict { paths: Vec<String> },
    Unsupported { operation: String },
}

pub enum Change {
    Added(Oid),
    RefMoved { name: String, from: Oid, to: Oid },
    LaneShifted(Oid),
}
```

4.3 `Unsupported` is the design, not a gap

The failure mode of a modelled git is not being wrong. It is being *confidently* wrong — producing a plausible graph that quietly differs from what the command will actually do, on exactly the operations a user cannot check by eye.

`Unsupported` makes that structurally impossible. If the plumbing cannot express an operation, the answer is "I cannot show you this", never a picture. A user who sees nothing goes and reads the docs; a user who sees a wrong picture acts on it.

This mirrors the existing sandbox posture: `INV-13`, there is no degraded mode.

5. The test that can go red — A5

The parity test is the one that earns the feature's trust, and it must be able to fail. For each applicable fixture repository:

1. Build the `Plan`.
2. `preview()` it — collect the predicted commit list and lane assignment.
3. Clone the fixture to a throwaway, **actually run the operation**, and lay out the real result.
4. Assert the two are identical.

A predicted graph that differs from the real one fails here, which is the only place the difference can be caught before a user sees it.

Mutation proof

Two mutations minimum, breaking differently, per the standing rule:

- **M1** — splice the hypothetical commit in at the wrong position. Expect `caught`.
- **M2** — return `Graph` where the code should return `Conflict`. Expect `caught`, on a different test, because a conflicting merge has no graph to be wrong about.

6. Out of scope, deliberately

- **Rebase, reset, force-push.** Expressible only with far more plumbing, and the owner's chosen first slice is the confusing operations rather than the destructive ones. They return `Unsupported`.
- **Animation.** git-school's charm is watching the graph move. That needs a before→after node correspondence — which node became which — and that is a design problem in its own right, not a rendering detail. Ship the marked diff first and find out whether the motion is missed.
- **The teaching sandbox.** Same function pointed at a scratch repo where mutation is allowed. It falls out of this work; it is not part of this slice.

7. Risks

Risk	Mitigation
<code>merge-tree --write-tree</code> needs <code>git ≥ 2.38</code>	Measured 2.43.0 here. <code>SUPPORTED_VERSIONS.md</code> floor is 2.32 — the preview must degrade to <code>Unsupported</code> below 2.38, not fail
The temp store grows on repeated previews	It is a temp dir per call, removed on drop. Asserted by test
Alternates could be pointed at a repo that moves	Resolve the object path once, at call time, and never cache it across calls
Preview drifts as the planner gains operations	<code>Unsupported</code> is the default arm, so a new operation is invisible rather than wrong

8. ADR

Yes — one is warranted. This decides a contract: **a preview is computed by real git against the real object store, and refuses rather than models.** That is expensive to reverse and easy for a later session to "optimise" into a simulation.

9. Open question for the owner

The first slice renders in the **web canvas**, because that renderer exists today. `#457`'s terminal renderer picks the same data up for free when it lands. Confirmed by the owner on 2026-08-29; recorded here because a future reader will wonder why a lazygit user got a web feature first.

Signed: max · 2026-08-29T09:30:00-04:00